



GameBot — Roadmap улучшений

Приоритеты

#	Фича	Сложность	Польза
1	Алёрты RPi (температура / undervoltage)	Низкая	🔥 Критично
2	История цен	Средняя	☆☆☆
3	Дайджест вместо спама	Низкая	☆☆☆
4	Расписание парсинга новостей	Минимальная	☆☆
5	Фильтр тегов для новостей	Средняя	☆☆
6	Еженедельный дайджест	Средняя	☆☆

1. 🧴 Алёрты RPi — температура и undervoltage

Зачем: автоматическое уведомление при перегреве или проблемах с питанием, не надо вручную жать `/status`.

Логика: фоновый таск каждые 5 минут проверяет состояние RPi.

```
TEMP_WARN = 65.0
TEMP_CRIT = 75.0
already_alerted = {"temp": False, "undervoltage": False}

async def check_health_alert(context):
    temp = get_cpu_temp()
    throttle = get_throttle_flags()

    if temp >= TEMP_CRIT and not already_alerted["temp"]:
        await context.bot.send_message(ADMIN_ID, f"🔴 КРИТИЧНО: температура {temp}")
        already_alerted["temp"] = True
    elif temp >= TEMP_WARN and not already_alerted["temp"]:
        await context.bot.send_message(ADMIN_ID, f"🟡 Предупреждение: температура {temp}")
        already_alerted["temp"] = True
```

```

elif temp < TEMP_WARN:
    already_alerted["temp"] = False # сброс флага

if throttle.get("undervoltage") and not already_alerted["undervoltage"]:
    await context.bot.send_message(ADMIN_ID, "⚡ Undervoltage detected! Проверь кабель")
    already_alerted["undervoltage"] = True
elif not throttle.get("undervoltage"):
    already_alerted["undervoltage"] = False

```

Подключение в main() :

```

application.job_queue.run_repeating(check_health_alert, interval=300, first=10)

```

Оценка времени: ~1-2 часа

2. История цен

Зачем: видеть динамику цены товара — “было 95k → 90k → 85k”.

Новая таблица в SQLite

```

CREATE TABLE IF NOT EXISTS price_history (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    item_id INTEGER NOT NULL,
    price INTEGER NOT NULL,
    found_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (item_id) REFERENCES monitored_items(id)
);

```

Запись при каждом парсинге

```

def save_price(item_id: int, price: int):
    conn.execute(
        "INSERT INTO price_history (item_id, price) VALUES (?, ?)",
        (item_id, price)
    )
    conn.commit()

```

Отображение в уведомлении

 Nintendo Switch найден!

💰 Цена: 85,000 ₴ (цель: 90,000 ₴ ✅)

📈 История: 95,000 → 90,000 → 85,000

🔗 [ссылка на объявление](#)

Кнопка "📈 История" в меню "Мои товары"

```
async def show_price_history(update, context, item_id):
    rows = conn.execute(
        "SELECT price, found_at FROM price_history WHERE item_id = ? ORDER BY found_at"
        "(item_id,)"
    ).fetchall()
    text = "\n".join([f"{r['price']:,} ₴ - {r['found_at'][:10]}" for r in rows])
    await update.callback_query.message.reply_text(f"📈 История цен:\n{text}")
```

Оценка времени: ~2-3 часа

3. 📦 Дайджест вместо спама

Зачем: если по запросу "Nintendo" появилось 8 объявлений подряд — не слать 8 сообщений.

Логика: буферизировать находки за интервал (30 минут), затем слать одним сообщением.

```
# Добавить колонку в таблицу товаров
ALTER TABLE found_items ADD COLUMN notified_at TIMESTAMP;

async def send_digest(context):
    unsent = conn.execute(
        "SELECT * FROM found_items WHERE notified_at IS NULL"
    ).fetchall()

    if not unsent:
        return

    # Группируем по запросу
    by_query = {}
    for item in unsent:
        by_query.setdefault(item["query"], []).append(item)

    for query, items in by_query.items():
        lines = [f"• {i['title']} - {i['price']:,} ₴ → {i['url']}" for i in items]
        text = f"📦 Новые объявления: {query} ({len(items)} шт.)\n\n" + "\n".join
```

```
await context.bot.send_message(ADMIN_ID, text, disable_web_page_preview=T

# Помечаем как отправленные
conn.execute("UPDATE found_items SET notified_at = CURRENT_TIMESTAMP WHERE no
conn.commit()
```

Подключение:

```
application.job_queue.run_repeating(send_digest, interval=1800) # каждые 30 мину
```

Оценка времени: ~1-2 часа

4. 🕒 Расписание парсинга новостей

Зачем: не гонять RSS непрерывно, разгрузить RPi. Новости приходят 3 раза в день батчами — удобнее для модерации.

```
from datetime import time
from zoneinfo import ZoneInfo

ALMATY_TZ = ZoneInfo("Asia/Almaty")

# В main() вместо run_repeating:
application.job_queue.run_daily(fetch_news, time=time(9, 0, tzinfo=ALMATY_TZ))
application.job_queue.run_daily(fetch_news, time=time(15, 0, tzinfo=ALMATY_TZ))
application.job_queue.run_daily(fetch_news, time=time(21, 0, tzinfo=ALMATY_TZ))
```

Оценка времени: ~30 минут

5. 🍷 Фильтр тегов для новостей

Зачем: автоматически скипать темы которые не интересны (mobile gaming, NFT, esports и т.п.) — меньше мусора на одобрение.

Таблица

```
CREATE TABLE IF NOT EXISTS blocked_tags (
    tag TEXT PRIMARY KEY
);
```

Команды

```
/blocktag mobile      — добавить тег в стоп-лист
/unblocktag mobile    — убрать тег из стоп-листа
/blockedtags          — показать все заблокированные теги
```

Изменение промпта Gemini

```
GEMINI_PROMPT = """
Перескажи новость на русском языке в стиле игрового журналиста.
Верни ТОЛЬКО JSON без markdown:
{
  "title": "заголовок",
  "summary": "текст новости",
  "tags": ["pc", "rpg", "indie"] // категории: pc, console, mobile, nft, esport
}
"""
```

Фильтрация перед отправкой на одобрение

```
blocked = {row["tag"] for row in conn.execute("SELECT tag FROM blocked_tags")}

result = call_gemini(article)
if any(tag in blocked for tag in result["tags"]):
    logging.info(f"Скип (заблокированный тег): {result['title']}")
    continue # не отправляем на одобрение
```

Оценка времени: ~2-3 часа

6. Еженедельный дайджест

Зачем: сводка активности за неделю — сколько новостей, алертов, как вёл себя RPi.

Таблица событий

```
CREATE TABLE IF NOT EXISTS events_log (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    event_type TEXT NOT NULL,      -- 'post_approved', 'post_rejected', 'price_alert
    meta TEXT,                    -- JSON с деталями
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Формат сообщения

 Итоги недели (24–30 марта):

 Новостей опубликовано: 23

 Одобрено: 23 |  Отклонено: 7

 Мониторится товаров: 5

 Алертов по ценам: 3

 Макс. температура RPi: 61°C

 Undervoltage за неделю: 0 раз

 Uptime: 7д 4ч 12м

Запуск

```
application.job_queue.run_daily(  
    send_weekly_digest,  
    time=time(20, 0, tzinfo=ALMATY_TZ),  
    days=(6,) # воскресенье  
)
```

Оценка времени: ~1-2 часа

Суммарно

Фича	Время
Алёрты RPi	~1-2 ч
История цен	~2-3 ч
Дайджест вместо спама	~1-2 ч
Расписание новостей	~30 мин
Фильтр тегов	~2-3 ч
Еженедельный дайджест	~1-2 ч
Итого	~8-13 ч

